

RAG NOTES

Retrieval-Augmented Generation — Student Reference Guide

VPro Skills | Advanced RAG with Claude, LangChain, ChromaDB and FastAPI

1. What is RAG?

Retrieval-Augmented Generation (RAG) is an architecture that allows a Large Language Model (LLM) to answer questions using information retrieved from an external knowledge source.

Traditional LLM

A normal LLM mainly relies on the knowledge contained in its training and the information provided in the current prompt.

RAG

RAG adds a retrieval step. The system first searches a knowledge base for relevant information and then gives that information to the LLM as context.

Simple flow: User Question → Retrieve Relevant Information → Add Context → LLM → Answer

Why RAG?

- Reduce hallucination by grounding answers in retrieved information.
- Use private or organization-specific documents.
- Update knowledge without retraining the LLM.
- Provide source information with answers.

2. RAG Architecture

Document Ingestion Pipeline

Documents are prepared before users ask questions.

PDF / TXT → Document Loader → Text Splitting → Embeddings → Vector Database

Query Pipeline

The query pipeline runs whenever a user asks a question.

Question → Query Processing → Retrieval → Context → LLM → Answer

Important Components

Component	Purpose
Document Loader	Reads PDFs, text files and other knowledge sources.
Text Splitter	Breaks large documents into smaller chunks.
Embedding Model	Converts text into numerical vectors.
Vector Database	Stores vectors and searches semantically similar content.
Retriever	Finds candidate chunks relevant to a question.
Reranker	Re-orders candidate chunks by relevance.
LLM	Uses retrieved context to generate the final response.

3. Naive RAG vs Advanced RAG

Naive RAG

A simple RAG implementation commonly performs one query, retrieves the top-k chunks, and sends those chunks directly to the LLM.

```
Question → Embedding/Search → Top-K Chunks → LLM → Answer
```

Limitations

- A single query may not capture all ways a question can be expressed.
- The first retrieved results may not be perfectly ranked.
- Too many irrelevant chunks can reduce answer quality.
- Poor chunking can lose important context.

Advanced RAG

Advanced RAG improves multiple stages of the pipeline. Common techniques include query expansion, multi-query retrieval, metadata filtering, hybrid retrieval, reranking, contextual compression, and better prompt construction.

```
Question → Query Expansion → Multiple Retrievals → Candidate Pool → Reranking → Top Chunks → LLM
```

Key idea

Retriever finds candidates. Reranker finds the best candidates. LLM generates the grounded answer.

4. Advanced RAG Techniques

Query Expansion

One user question is converted into multiple search queries. This can improve retrieval recall because different queries may match different parts of the knowledge base.

Example: "What is Advanced RAG?" → "Advanced Retrieval Augmented Generation", "Advanced RAG architecture", "Advanced RAG retrieval techniques"

Multi-Query Retrieval

Each generated query is sent to the vector database. The results are combined and duplicate chunks are removed.

Re-Ranking

A first-stage vector search is fast and retrieves candidate documents. A Cross-Encoder can then score the question-document pairs more precisely and return the best results.

Vector Search → 20 candidates → Cross-Encoder → Top 5

Context Preparation

Only the most relevant chunks should be sent to the LLM. Context should include useful metadata such as source filename and page number when available.

5. Embeddings and ChromaDB

What is an embedding?

An embedding represents text as a numerical vector. Texts with similar meanings tend to have vectors that are close in the embedding space.

```
"Python is a programming language" → [0.21, -0.45, 0.73, ...]
```

Why embeddings?

Keyword search looks mainly for matching words. Semantic vector search can find content with similar meaning even when the exact words are different.

ChromaDB

ChromaDB is a vector database used to store document chunks, embeddings, and metadata and to retrieve semantically similar chunks.

Typical flow

```
Chunk → Embedding Model → Vector → ChromaDB
```

```
Question → Embedding Model → Similarity Search → Relevant Chunks
```

Metadata

Metadata can store source filename, page number, document type, department, date, or other useful attributes. Metadata can later support filtering and source citations.

6. Claude in Advanced RAG

In this project, Claude is used as the main Large Language Model through LangChain's Anthropic integration.

Claude Job 1 — Query Expansion

Claude receives the original question and creates alternative search queries.

```
Question → Claude → Query 1 / Query 2 / Query 3
```

Claude Job 2 — Answer Generation

Claude receives the user's question together with the highest-ranked retrieved context and generates a grounded answer.

```
Question + Retrieved Context → Claude → Final Answer
```

Grounding instruction

A useful RAG prompt should instruct the model to use the supplied context and avoid inventing information that is not supported by the retrieved documents.

7. LangChain Components Used

langchain

Provides common abstractions for documents, prompts, models, retrievers and application workflows.

langchain-community

Provides community-maintained integrations such as document loaders.

langchain-text-splitters

Provides text splitting utilities such as RecursiveCharacterTextSplitter.

langchain-huggingface

Connects LangChain applications with Hugging Face embedding models.

langchain-chroma

Connects LangChain with ChromaDB.

langchain-anthropic

Connects LangChain applications with Anthropic Claude models.

Memory trick: LangChain connects the components; specialized packages connect LangChain to specific technologies.

8. FastAPI Integration

FastAPI is the API layer around our RAG pipeline. It allows a frontend, Postman, mobile application, or another service to communicate with the RAG system.

Endpoints

POST /upload → Upload and index a PDF or TXT document

POST /ask → Ask a question and receive an answer with sources

Complete application flow

Client → FastAPI → Advanced RAG → Claude/ChromaDB → FastAPI → Client

Swagger UI

FastAPI automatically provides interactive API documentation at /docs, making it convenient for classroom demonstrations and testing.

9. Sample Questions for Classroom Testing

Use these questions after uploading this document into the Advanced RAG application.

1. What is RAG?
2. Why is RAG useful?
3. What is the difference between Naive RAG and Advanced RAG?
4. What is query expansion?
5. Why do we use multiple queries?
6. What is re-ranking?
7. Why is a Cross-Encoder used after vector search?
8. What is an embedding?
9. What is the role of ChromaDB?
10. What is the difference between an embedding model and an LLM?
11. How is Claude used in this Advanced RAG project?
12. What is the role of FastAPI?
13. What information can be stored in document metadata?
14. Explain the complete Advanced RAG pipeline.

Expected Advanced RAG Pipeline

Documents → Load → Chunk → Embed → Store → Query Expansion → Retrieve → Deduplicate → Re-rank → Context → Claude → Answer + Sources

VPro Skills — Learn. Build. Succeed.